

ActSWM: Action-Sensitive World Models for Long-Horizon Planning in Open-World Games

Zhenfeng Gan, ZiTong Zeng, Jiajun Cheng, Yeke Song,
Yongyi Tang*, Xueqian Wang*

*Corresponding authors

Abstract

Latent world models support efficient model-predictive control by optimizing future control sequences in latent space and replanning in a receding-horizon manner. However, existing latent predictors often lack stable long-horizon rollout ability, and prediction accuracy alone does not ensure that rollouts remain responsive to the actions being planned. We identify *Context Collapse*, a failure mode in which autoregressive latent predictors maintain high similarity to future states while producing nearly indistinguishable futures under different action sequences. To address this issue, we propose ActSWM, an action-sensitive latent world model grounded in a transition-separation principle: a planning-useful latent dynamics model should keep alternative-action futures distinguishable and make the action associated with each local transition recoverable. Under this principle, action sensitivity is enforced as a constraint on latent rollouts rather than treated only as an auxiliary prediction target, encouraging predicted futures to preserve action-dependent differences over long horizons. Across step-drift analysis, closed-loop Minecraft planning, and cross-game local action recovery, ActSWM preserves larger action-dependent rollout gaps than existing baselines, improves task success in long-horizon interactive settings, and enables world-model-based action recovery from offline gameplay videos.

Introduction

Building general-purpose autonomous agents that can perceive, reason, and act in open worlds has long been a central objective in artificial general intelligence (AGI) research. Open-world games provide a concrete testbed for this objective because they combine high-dimensional visual perception, low-level control, and extended task structure. In this setting, recent game agents have made substantial progress: behavior-cloning approaches such as Video Pretraining (VPT) and STEVE-1 learn policies from large-scale gameplay videos (Baker et al. 2022; Lifshitz et al. 2023); multimodal agent systems, including the JARVIS family and Game-TARS, integrate visual perception, language-based reasoning, and low-level control for open-world decision making (Wang et al. 2025a, 2024, 2025b); and Lumine has demonstrated real-time, hours-long task completion in complex 3D open-world environments (Tan et al. 2025). Despite these advances, existing methods still face three persistent limitations: low sample efficiency, high deployment

cost, and insufficient long-horizon decision-making ability. These limitations become especially pronounced in open-ended tasks that require extended action sequences, such as navigation, resource gathering, object manipulation, and construction (Hafner 2021; Fan et al. 2022; Wang et al. 2023). In such settings, an agent must not only interpret the current visual state, but also anticipate how actions taken now may cascade into different future states over many time steps.

World-model-based planning offers an attractive route toward addressing these challenges. By learning environment dynamics and optimizing future control sequences in latent space, an agent can evaluate action consequences with substantially fewer real interactions (Hafner et al. 2024; Hansen, Su, and Wang 2024). Recent Joint-Embedding Predictive Architecture (JEPA) methods further suggest that predicting in latent space can improve the efficiency and abstraction of world-model learning (Assran et al. 2023; Bardes et al. 2024). LeWorldModel (LeWM) applies this idea to lightweight, plannable latent dynamics learned end-to-end from raw pixels (Maes et al. 2026). These methods highlight the promise of latent world models for efficient planning and make action sensitivity a central requirement: action conditioning is useful only if distinct action sequences induce separable latent consequences. A latent predictor optimized mainly for future-state similarity does not necessarily satisfy this requirement. The World-Action Model (WAM) augments DreamerV2 with an inverse-dynamics head trained jointly on consecutive encoder embeddings, encouraging the representation to retain action-relevant information (Han and Yilmaz 2026). However, this objective operates on observed local transitions and does not explicitly constrain predicted rollouts under alternative future action sequences.

We therefore study a rollout-level failure mode that is distinct from the standard accumulation of prediction error in long-horizon world models (Bengio et al. 2015; Zhang et al. 2026a; Chen et al. 2024). In this failure mode, degradation arises because action conditioning itself collapses: as the latent context becomes more informative, the predictor can extrapolate state progression from visual history while becoming insensitive to the conditioning action sequence. We refer to this phenomenon as *Context Collapse*. In this setting, longer contexts or multi-step rollout objectives may improve similarity to encoded future states, yet predictions conditioned on recorded actions can remain nearly identical to

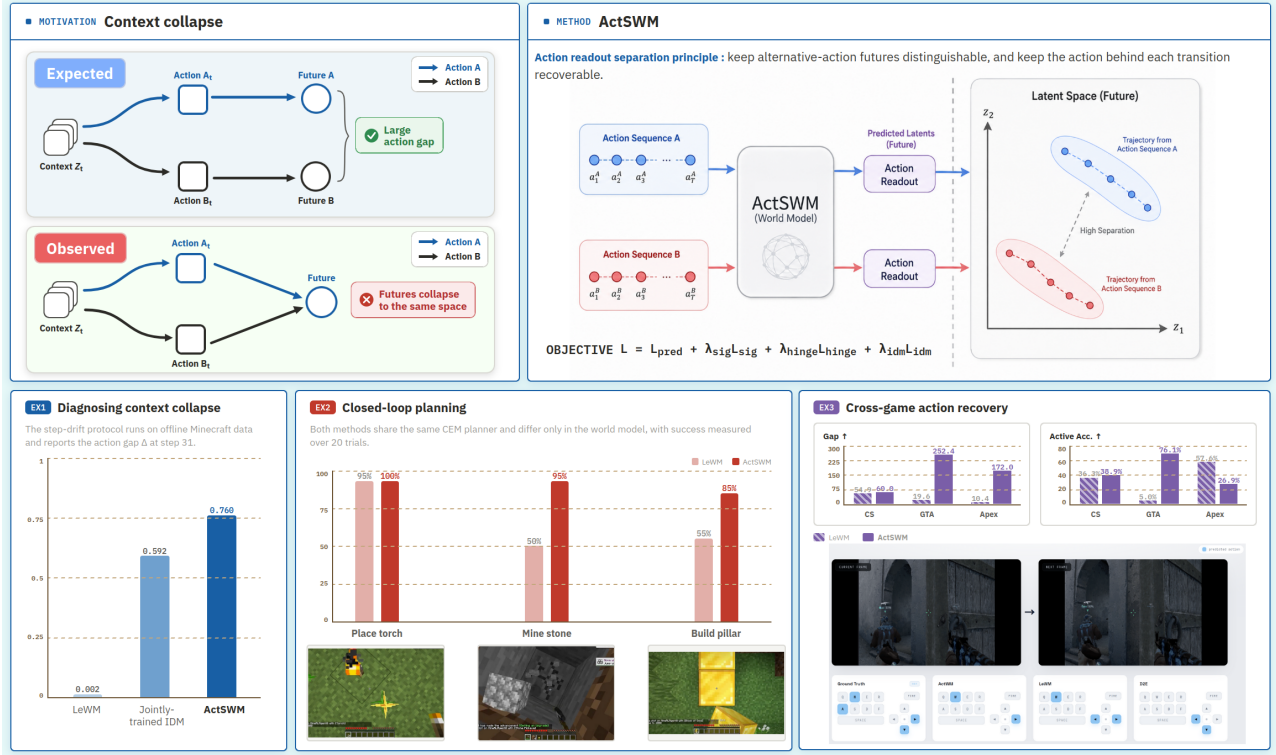


Figure 1: Overview of ActSWM. Latent world models can suffer from context collapse, where long-horizon rollouts match plausible futures but become insensitive to action inputs. ActSWM addresses this by enforcing action-sensitive latent rollouts, leading to larger action gaps, stronger closed-loop planning, and larger CEM-over-random gaps in cross-game local action recovery.

those conditioned on alternatives such as all-zero actions. For planning-oriented world models, this is particularly harmful: if distinct action sequences fail to induce distinguishable latent futures, a planner cannot reliably compare their downstream consequences.

To address this problem, ActSWM explicitly trains predicted latent rollouts to remain sensitive to the conditioning action sequence. As a result, the predicted futures encode action-dependent state changes rather than merely matching likely future states. This design preserves the efficiency of latent prediction while discouraging action-agnostic rollouts. Figure 1 gives an overview of the motivation, method, and evaluation protocol.

Our main contributions are:

- We introduce **ActSWM**, a JEPA-based latent world model designed to support both **long-horizon prediction** and **action-sensitive rollout dynamics**.
- We validate latent world-model planning in open-ended sandbox-game tasks. In MineStudio closed-loop planning, ActSWM improves success over LeWM by up to **45%** on stone mining and **30%** on pillar building.
- To diagnose Context Collapse, we compare long-horizon rollouts conditioned on recorded and zero actions. ActSWM produces a roughly **380×** larger separation between the resulting futures than the strongest baseline,

showing that its rollouts remain responsive to action inputs.

- To test whether the learned dynamics can recover controls from offline videos, we use CEM to infer action sequences that reproduce target future frames across three domains. ActSWM achieves up to a **16.5×** larger cost improvement over random search and improves **accuracy** by up to **0.711** in absolute terms.

Related Work

Game Agents

Game environments are widely used for studying general-purpose agents because they combine visual perception, long-horizon interaction, and reproducible evaluation. Beyond policy pretraining, several lines of work have turned games into broad testbeds for open-ended agent learning. Crafter evaluates diverse survival-game abilities through semantically meaningful achievements, while MineDojo expands Minecraft into a large task suite with internet-scale multimodal knowledge (Hafner 2021; Fan et al. 2022). Another line studies hierarchical decision making: Plan4MC learns reusable Minecraft skills and plans over a skill graph, and Voyager uses an LLM-driven curriculum with an expanding skill library for open-ended exploration (Yuan et al. 2023; Wang et al. 2023). These works emphasize benchmark

construction, external knowledge, high-level skill planning, or language-code agents. Our focus is complementary: we study whether a compact latent world model can preserve action-dependent dynamics so that downstream planners can compare candidate futures reliably.

Latent World Models and Planning

Latent world models support sample-efficient control by learning compact dynamics for planning or policy optimization. PlaNet, Dreamer, and the TD-MPC family combine learned latent transitions with online planning, actor-critic learning, or model predictive control (Hafner et al. 2019, 2024; Hansen, Wang, and Su 2022; Hansen, Su, and Wang 2024). DIAMOND emphasizes visual fidelity, DINO-WM supports zero-shot goal reaching, and LeWM learns lightweight JEPA dynamics from pixels (Alonso et al. 2024; Zhou et al. 2025; Maes et al. 2026). Long-horizon errors arising from exposure bias and accumulated rollout drift have motivated diffusion correction, hierarchical or variable-length prediction, temporal discrimination, and improved transformer training (Bengio et al. 2015; Chen et al. 2024; Zhang et al. 2026a; Du et al. 2026; Peng and Molin 2026; Dedieu et al. 2025). LS-Imagine further extends imagination for open-world exploration in MineDojo (Li et al. 2026). These methods improve temporal prediction or policy learning, whereas Context Collapse concerns a distinct axis: accurate rollouts may still respond weakly to alternative actions.

Action Representation in Latent Dynamics

For a world model to support planning, actions must be represented through their effects on latent transitions. Latent-action methods learn controls from passive video or jointly co-evolve latent actions with generative world models, enabling unlabeled data use but introducing an inferred action space (Alles et al. 2025; Wang et al. 2026). WAM, Sensorimotor World Models, and actionable process world models instead use inverse or bidirectional action prediction to preserve locally controllable factors (Han and Yilmaz 2026; Ivashkov, Balestrieri, and Schölkopf 2026; Yan et al. 2025). Delta-JEPA decodes actions from latent displacements, while DWM separates action-invariant world effects from action-driven residuals (Zhang et al. 2026c,b). World2Act contrastively aligns policy actions with world-model dynamics for VLA post-training rather than constraining the world model itself (Vuong et al. 2026). These approaches establish the value of action-aware geometry, while leaving open how to jointly constrain local transitions and multi-step counterfactual futures.

Method

This section introduces the ActSWM architecture and training objective for learning action-sensitive latent dynamics. ActSWM combines multi-step latent prediction with rollout separation and a fixed action readout that constrains latent transitions to retain recoverable action information.

Preliminaries

Given an offline trajectory of observations and actions $\tau = \{(o_t, a_t)\}_{t=1}^T$, our goal is to learn a compact dynamics model

that predicts future embeddings rather than reconstructing future pixels. A visual encoder maps observations to $z_t = f_\theta(o_t)$, and an action encoder maps actions to $e_t = g_\theta(a_t)$. The predictor p_θ takes a window of encoded observations and action features as input and estimates the next latent state:

$$\hat{z}_{t+1} = p_\theta(z_{t-H+1:t}, e_{t-H+1:t}), \quad (1)$$

where θ collectively denotes the trainable world-model parameters, H is the context length, and t denotes the final time step in the context window. For multi-step prediction, the same predictor is applied recursively under future action sequences, producing latent trajectories for downstream evaluation.

Modeling Action Sensitivity

The formulation above specifies how actions condition the predictor, but conditioning alone does not ensure that distinct action sequences induce distinct rollouts. Action-sensitive latent dynamics should produce distinguishable future predictions whenever different control sequences imply different consequences. We therefore separate action sensitivity into a rollout-level criterion and a transition-level criterion.

At the rollout level, alternative future action sequences starting from the same context should lead to separable predicted futures. For an arbitrary horizon K , let $\mathbf{a}_{t:t+K-1}^{(1)}$ and $\mathbf{a}_{t:t+K-1}^{(2)}$ denote two future action sequences, and let $\hat{z}_{t+1:t+K}^{(1)}$ and $\hat{z}_{t+1:t+K}^{(2)}$ denote their corresponding K -step rollouts from the same context. An action-sensitive world model should keep these futures distinguishable whenever the two sequences imply different consequences. In our training and evaluation, we use the recorded future action sequence and an all-zero sequence as a controlled contrast for measuring this property.

At the transition level, the input action should be recoverable from the change between adjacent latent states. Specifically, a transition $u_t = [z_t, z_{t+1}]$ should contain sufficient information to infer a_t , where $[\cdot, \cdot]$ denotes feature concatenation. This criterion is related to inverse-dynamics and bidirectional action-prediction regularization (Ivashkov, Balestrieri, and Schölkopf 2026; Yan et al. 2025), while rollout-level separation additionally tests whether action-dependent differences persist over multiple predicted steps.

Action-Readout Separation Principle

We implement the transition-level criterion with an action readout. Let q_ϕ map latent transitions to actions. If the same readout can recover different actions from different latent transitions, then those transitions must be distinguishable in latent space.

This readout-based constraint can be stated in terms of transition separation. Suppose q_ϕ is locally L -Lipschitz and two transitions u_i, u_j are decoded with errors ϵ_i and ϵ_j , i.e.,

$$\|q_\phi(u_i) - a_i\| \leq \epsilon_i, \quad \|q_\phi(u_j) - a_j\| \leq \epsilon_j. \quad (2)$$

Then the triangle inequality and Lipschitz continuity imply

$$\|u_i - u_j\| \geq \frac{\|a_i - a_j\| - \epsilon_i - \epsilon_j}{L}. \quad (3)$$

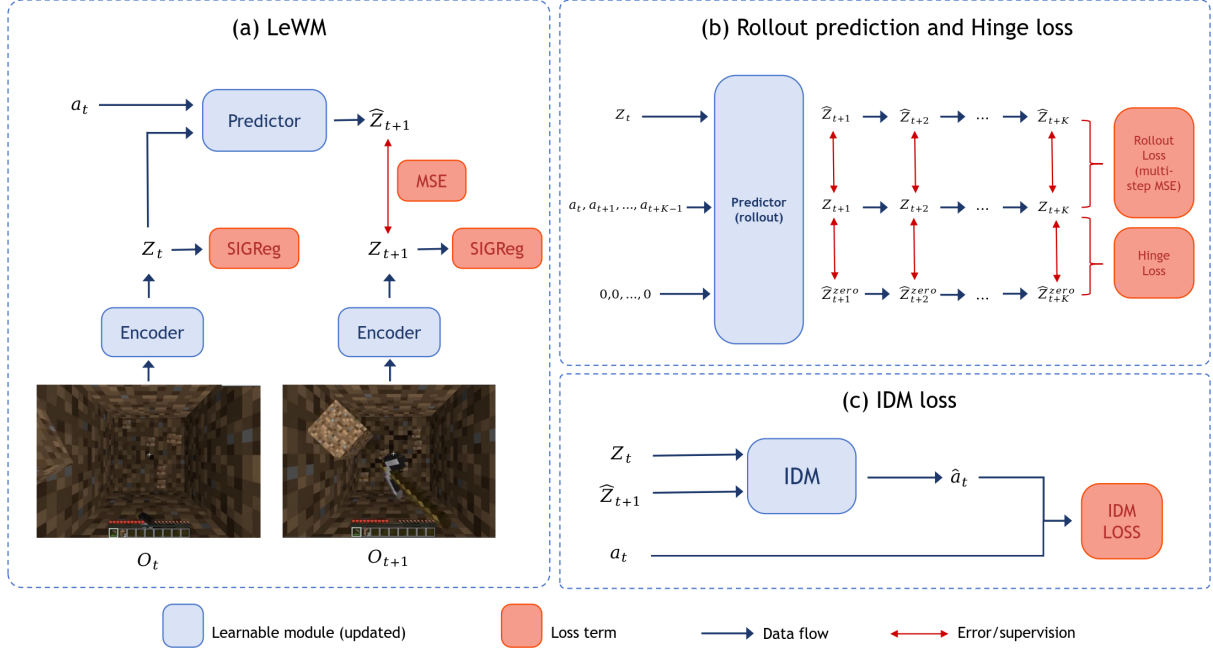


Figure 2: ActSWM framework. ActSWM combines JEPA-based latent prediction, multi-step rollout training, an action-sensitivity hinge loss, and a fixed action readout to learn action-sensitive latent dynamics.

Thus, if different actions are recovered accurately by a shared readout, their latent transitions must remain separated by a positive margin whenever $\|a_i - a_j\| > \epsilon_i + \epsilon_j$. If the readout changes together with the representation, however, the action-recovery loss can decrease by moving the readout boundary rather than by increasing transition separation. ActSWM therefore implements this principle with a parameter-frozen readout, introduced next. Appendix E gives the complete derivation.

ActSWM Architecture

ActSWM builds on the latent prediction backbone defined above, but augments it with two action-sensitivity mechanisms. The rollout branch compares futures generated by recorded and alternative actions, while the readout branch uses a randomly initialized action readout whose parameters remain frozen during world-model training. The backbone retains a shared prediction space for multi-step prediction and representation regularization. Figure 2 illustrates the model architecture.

For an adjacent latent transition $u_t = [z_t, z_{t+1}]$, the action readout q_{ϕ_0} predicts the corresponding action a_t , where ϕ_0 denotes its frozen parameters. During training, ϕ_0 is excluded from optimization, but the readout loss still propagates through its latent inputs. It therefore updates the encoder for encoded transitions and both the encoder and predictor for rollout-predicted transitions. In this way, the fixed readout implements the transition-separation principle above as a constraint on latent transition representations rather than as an adaptive auxiliary decoder.

Training Objective

ActSWM is trained to enforce the rollout-level and transition-level criteria while preserving latent-space prediction fidelity. Given a training segment of length $H + K$ whose context window ends at t , the model encodes $z_{t-H+1:t}$ as context and autoregressively predicts the next K latent embeddings using the recorded future actions, producing $\hat{z}_{t+1:t+K}^{\text{gt}}$. Here, K denotes the training rollout horizon. This gives the multi-step prediction loss

$$\mathcal{L}_{\text{pred}} = \frac{1}{K} \sum_{k=1}^K \|\hat{z}_{t+k}^{\text{gt}} - z_{t+k}\|_2^2. \quad (4)$$

This prediction term aligns the rollout conditioned on recorded actions with the encoded future trajectory, but by itself it does not enforce the rollout-level criterion defined above. We therefore add a contrastive rollout from the same initial context, replacing the future actions with zeros to obtain $\hat{z}_{t+1:t+K}^0$. The hinge loss then penalizes excessive similarity between the futures produced under recorded and all-zero actions:

$$\ell_k = \max(0, \cos(\hat{z}_{t+k}^{\text{gt}}, \hat{z}_{t+k}^0) - (1 - m)), \quad (5a)$$

$$\mathcal{L}_{\text{hinge}} = \frac{1}{K} \sum_{k=1}^K \ell_k, \quad (5b)$$

where $\cos(\cdot, \cdot)$ denotes cosine similarity and m is the hinge-margin hyperparameter. This action-contrastive hinge term encourages the predicted futures to remain separable under distinct action conditions. Unlike cross-modal policy alignment in World2Act (Vuong et al. 2026), this contrast is ap-

plied within the world model between futures generated from the same context under different controls.

For the transition-level criterion, we apply the frozen readout to both the encoded transition and the rollout-predicted transition associated with the same action. To express the predicted rollout uniformly from its observed starting point, define

$$\tilde{z}_{t+k} = \begin{cases} z_t, & k = 0, \\ \hat{z}_{t+k}^{\text{gt}}, & 1 \leq k \leq K. \end{cases} \quad (6)$$

For $k = 0, \dots, K-1$, the encoded and predicted transitions are

$$u_{t+k}^{\text{enc}} = [z_{t+k}, z_{t+k+1}], \quad u_{t+k}^{\text{pred}} = [\tilde{z}_{t+k}, \tilde{z}_{t+k+1}]. \quad (7)$$

With the frozen action readout q_{ϕ_0} , the readout loss is

$$\ell_k^{\text{enc}} = \|q_{\phi_0}(u_{t+k}^{\text{enc}}) - a_{t+k}\|_2^2, \quad (8a)$$

$$\ell_k^{\text{pred}} = \|q_{\phi_0}(u_{t+k}^{\text{pred}}) - a_{t+k}\|_2^2, \quad (8b)$$

$$\mathcal{L}_{\text{readout}} = \frac{1}{K} \sum_{k=0}^{K-1} (\ell_k^{\text{enc}} + \alpha_{\text{pred}} \ell_k^{\text{pred}}), \quad (8c)$$

where $\alpha_{\text{pred}} = 1$ in the default configuration. The first term aligns encoded transitions with the recorded action, and the second applies the same constraint to rollout transitions. Since q_{ϕ_0} is fixed, gradients from this loss are propagated only through its latent inputs, thereby updating the encoder and predictor but not the readout parameters. The full training objective is

$$\mathcal{L} = \mathcal{L}_{\text{pred}} + \lambda_{\text{sig}} \mathcal{L}_{\text{sig}} + \lambda_{\text{hinge}} \mathcal{L}_{\text{hinge}} + \lambda_{\text{readout}} \mathcal{L}_{\text{readout}}, \quad (9)$$

where $\mathcal{L}_{\text{sig}}$ denotes the SigReg loss adopted from LeWM (Maes et al. 2026), which stabilizes the scale and distribution of prediction-space features, and the λ coefficients weight the corresponding objectives. Together, these terms optimize prediction fidelity, alternative-action rollout separation, and action-discriminative transition representations. Training configurations, loss weights, and implementation details are provided in Appendix A.

Experiments

We organize the experiments around three questions:

Q1: Does ActSWM mitigate Context Collapse? We test whether predicted rollouts remain distinguishable under different action sequences. Specifically, we compare rollouts conditioned on recorded and all-zero actions to determine whether the model relies on action inputs rather than context alone.

Q2: Does action-sensitive latent prediction improve closed-loop planning? We evaluate whether a model that better separates action consequences in latent space can guide closed-loop control optimization more reliably in interactive tasks.

Q3: Are the learned action representations more discriminative? We evaluate local action recovery using the CEM-over-random action gap, overall key accuracy, and active-key accuracy on offline gameplan videos.

Q1: Diagnosing Context Collapse

Setup and variants. The evaluation uses offline Minecraft H5 trajectories from VPT (Baker et al. 2022), where each window provides time-aligned image frames and low-level actions. The main setting uses context length $H = 32$, rollout horizon $K_{\text{eval}} = 32$, 128px inputs, and a frameskip of 4. We compare LeWM variants with short context ($H = 3$), 256px inputs, and multi-step rollout training, together with a jointly trained action-readout variant and ActSWM with its fixed readout. All variants use the same offline step-drift protocol and are evaluated under both recorded and all-zero future actions; no environment interaction is involved.

Metrics. For each sampled window, the first H steps are encoded as context. Let K_{eval} denote the evaluation rollout horizon. From the same context, we perform two K_{eval} -step latent rollouts: one conditioned on the recorded future action sequence, producing $\hat{z}_{t+1:t+K_{\text{eval}}}^{\text{gt}}$, and another conditioned on an all-zero action sequence, producing $\hat{z}_{t+1:t+K_{\text{eval}}}^0$. For each $k \in \{1, \dots, K_{\text{eval}}\}$, we compute the three quantities in Eq. 10:

$$s_k^{\text{gt}} = \cos(\hat{z}_{t+k}^{\text{gt}}, z_{t+k}), \quad (10a)$$

$$s_k^0 = \cos(\hat{z}_{t+k}^0, z_{t+k}), \quad (10b)$$

$$\Delta_k = s_k^{\text{gt}} - s_k^0. \quad (10c)$$

Here s_k^{gt} measures similarity under the recorded actions, s_k^0 measures similarity under all-zero actions, and Δ_k is the action gap. Context Collapse is indicated by high s_k^{gt} but a small Δ_k : the model predicts plausible futures while failing to make actions change those futures. The complete step-drift protocol restates these metrics in Appendix B.

Results. Longer context and multi-step training improve prediction under recorded actions but leave the rollouts conditioned on recorded and zero actions nearly indistinguishable. A jointly trained readout increases separation at the cost of prediction quality, whereas ActSWM retains high fidelity, reduces similarity under zero actions by over 80%, and achieves the largest action gap.

Q2: Closed-Loop Model-Based Planning

Task setup. We test whether the action-sensitive dynamics from Q1 improve closed-loop planning under the same training data and planner. MineStudio evaluation covers torch placement, stone mining, and pillar building, spanning short interactions, sustained control, and sequential actions (Cai et al. 2025). Related open-world systems evaluate policy learning in MineDojo or Craftax (Li et al. 2026; Dedieu et al. 2025); our reference-tracking protocol instead uses LeWM as the matched backbone baseline. Planning targets are encoded from task-specific reference trajectories described in Appendix C.

Planning objective. The encoded reference trajectory provides a sequence of visual goals for model predictive control (MPC). Let K_{plan} denote the planning horizon. At each environment step, given the current context and a reference future latent sequence $z_{t+1:t+K_{\text{plan}}}^{\text{goal}}$, the planner searches over an action-block sequence $\mathbf{a}_{t:t+K_{\text{plan}}-1}$ of length K_{plan} . For any

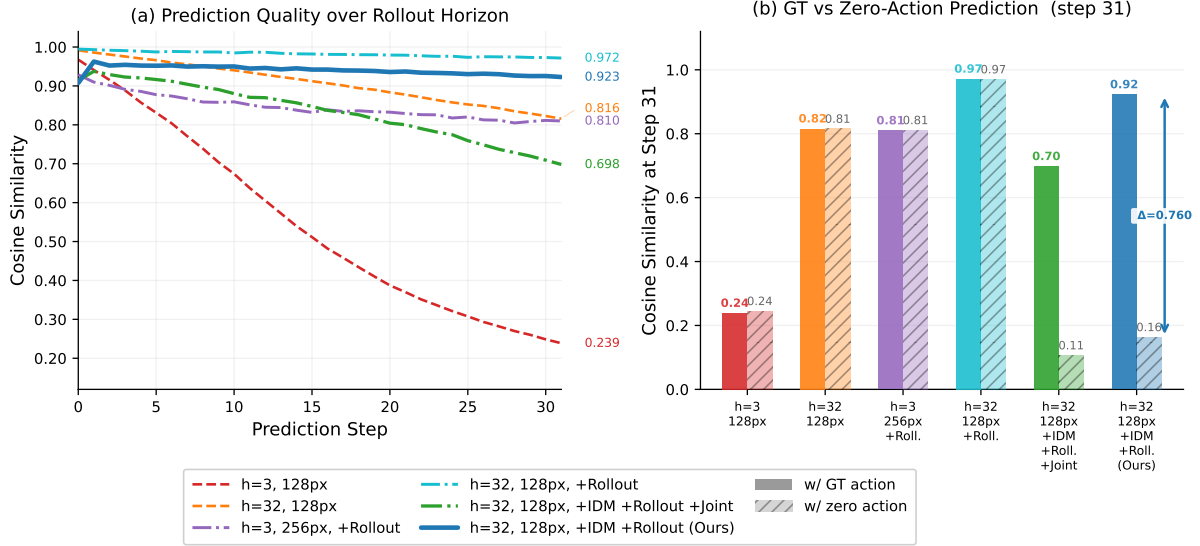


Figure 3: Q1 offline step-drift diagnostic on Minecraft VPT trajectories. The left panel tracks recorded-action prediction quality over a 32-step rollout. The right panel compares recorded-action and all-zero-action similarity at step 31 for the context, resolution, rollout-training, and readout variants described in the text. A larger gap indicates stronger action sensitivity.

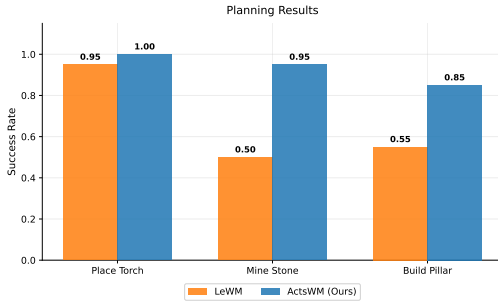


Figure 4: Success-rate comparison on Minecraft planning tasks. ActSWM outperforms LeWM on placing a torch, mining a stone block, and building a pillar.

candidate sequence, the world model rolls out K_{plan} steps and predicts $\hat{z}_{t+1:t+K_{\text{plan}}}(\mathbf{a})$. We define the planning cost as the average latent distance to the reference future:

$$J(\mathbf{a}) = \frac{1}{K_{\text{plan}}} \sum_{k=1}^{K_{\text{plan}}} \left\| \hat{z}_{t+k}(\mathbf{a}) - z_{t+k}^{\text{goal}} \right\|_2. \quad (11)$$

The optimized action sequence is

$$\mathbf{a}^* = \arg \min_{\mathbf{a}_{t:t+K_{\text{plan}}-1}} J(\mathbf{a}). \quad (12)$$

We use CEM to minimize J , execute the first optimized action block, and replan after the next observation (de Boer et al.). All models share the action space, targets, optimizer, and execution rule; we report success over 20 trials. Full planner settings and success criteria are given in Appendix C. **Results.** Relative to the baseline, ActSWM improves success by 5.3% on torch placement, 90.0% on stone mining,

and 54.5% on pillar building. The larger gains on sustained and sequential tasks indicate that action-sensitive rollouts improve planning when success depends on distinguishing long action sequences.

Q3: Discriminative Action Representations

Evaluation setup. We test whether latent dynamics can recover the actions explaining transitions in offline videos from Counter-Strike 2 (CS), Grand Theft Auto V (GTA), and Apex Legends (Apex). These games cover distinct visual layouts, motion patterns, and interaction rhythms, providing a diverse evaluation of action recovery. Given context and a target frame, CEM searches for an action sequence whose rollout approaches the target, providing an open-loop measure of recoverable action information. We compare direct inverse-dynamics prediction with CEM recovery using LeWM and ActSWM. Latent-action methods such as CoLA-World and LAWM infer task-specific action spaces and therefore are not directly scored against the fixed 14-control vocabulary used here (Wang et al. 2026; Alles et al. 2025). Data processing, context lengths, and the stepwise CEM procedure are detailed in Appendices D and F.

After constructing the full $K_{\text{rec}} = 32$ -step sequence, we compute the local recovery cost by rolling out the world model and comparing the final predicted latent with the target-frame latent:

$$J_{\text{local}}(\hat{\mathbf{a}}) = \left\| \hat{z}_{t+K_{\text{rec}}}(\hat{\mathbf{a}}) - z_{t+\delta} \right\|_2^2. \quad (13)$$

Metrics. We report three metrics.

Action Gap. We compare the CEM recovery cost with random sequences sampled from observed actions:

$$\text{Gap} = J_{\text{rand}} - J_{\text{cem}}, \quad (14)$$

Algorithm 1: Stepwise CEM for Local Action Recovery

Require: World model M , context latents $\mathbf{z}_{t-H+1:t}$, target latent $z_{t+\delta}$, action pool $\mathcal{A}$, GT statistics $\mu_{\text{gt}}, \sigma_{\text{gt}}$, horizon K , CEM params (N_c, N_e, N_i)

Ensure: Recovered action sequence $\hat{\mathbf{a}}_{1:K}$

```

1:  $\mathbf{b} \leftarrow [a_{t-H+1}, \dots, a_t]$  {action buffer from GT context}
2:  $\mathbf{e} \leftarrow \mathbf{z}_{t-H+1:t}$  {latent state}
3: for  $k = 1$  to  $K$  do
4:   if  $k = 1$  then
5:      $\mu \leftarrow \mu_{\text{gt}}, \sigma \leftarrow \sigma_{\text{gt}}$ 
6:   else
7:      $\mu \leftarrow \hat{a}_{k-1}, \sigma \leftarrow 0.5\sigma_{\text{gt}}$ 
8:   end if
9:   for  $i = 1$  to  $N_i$  do
10:    if  $i = 1$  then
11:      Sample  $\lfloor N_c/4 \rfloor$  candidates from  $\mathcal{A}$  and the rest from  $\mathcal{N}(\mu, \sigma)$ 
12:    else
13:      Sample  $N_c$  candidates from  $\mathcal{N}(\mu, \sigma)$ 
14:    end if
15:    for each candidate  $c$  do
16:      Build 56-dim input:  $\mathbf{b}[-3:] \oplus c$ 
17:       $\hat{z} \leftarrow M.\text{predict}(\mathbf{e}, \text{action})$ 
18:       $\text{cost}(c) \leftarrow \|\hat{z} - z_{t+\delta}\|_2^2$ 
19:    end for
20:     $\mathcal{E} \leftarrow \text{top-}N_e \text{ candidates by lowest cost}$ 
21:     $\mu \leftarrow \text{mean}(\mathcal{E}), \sigma \leftarrow \text{std}(\mathcal{E})$ 
22:  end for
23:   $\hat{a}_k \leftarrow \mu$ 
24:   $\mathbf{e} \leftarrow M.\text{predict}(\mathbf{e}, \hat{a}_k)$ ; append  $\hat{a}_k$  to  $\mathbf{b}$ 
25: end for
26: return  $\hat{\mathbf{a}}_{1:K}$ 

```

where J_{rand} is the random-sequence rollout cost. A large positive value indicates that the model can steer its latent rollout toward the target.

Key Accuracy. We compute per-frame binary key accuracy over all 14 key dimensions. Let $b(x) = \mathbf{1}(x > 0.5)$ denote thresholding. Then

$$c_{i,k,d} = \mathbf{1}[b(\hat{a}_{i,k,d}) = b(a_{i,k,d})], \quad (15a)$$

$$\text{Acc}_{\text{key}} = \frac{1}{NK_{\text{rec}}D_a} \sum_{i,k,d} c_{i,k,d}, \quad (15b)$$

where $c_{i,k,d}$ indicates a correct control prediction, with sums over $i = 1, \dots, N$, $k = 0, \dots, K_{\text{rec}} - 1$, and $d = 1, \dots, D_a$. Here $N = 15$ is the number of evaluation windows per game and $D_a = 14$ is the number of binary control dimensions.

Active Accuracy. Key accuracy alone can be inflated by correctly predicting the dominant zero class for rarely pressed keys. We therefore also report active accuracy, which evaluates recall only on frames where the ground-truth key is active:

$$\text{Acc}_{\text{active}} = \frac{\sum_{i,k,d} \mathbf{1}[b(\hat{a}_{i,k,d}) = 1 \wedge b(a_{i,k,d}) = 1]}{\sum_{i,k,d} \mathbf{1}[b(a_{i,k,d}) = 1]}. \quad (16)$$

This metric directly measures whether the model recovers actions when the human player is actually pressing keys.

Table 1: Local action-recovery results over 14 binary controls. Gap measures the improvement of CEM over random search. G-IDM predicts actions directly and therefore has no Gap.

Game	Method	Gap $\uparrow$	Key Acc. $\uparrow$	Active Acc. $\uparrow$
CS	D2E G-IDM	–	0.913	0.000
CS	LeWM	54.86	0.703	0.363
CS	ActSWM (Ours)	60.02	0.743	0.389
GTA	D2E G-IDM	–	0.709	0.000
GTA	LeWM	19.57	0.684	0.050
GTA	ActSWM (Ours)	252.38	0.662	0.761
Apex	D2E G-IDM	–	0.876	0.000
Apex	LeWM	10.44	0.639	0.576
Apex	ActSWM (Ours)	172.01	0.721	0.269

Since G-IDM directly outputs action predictions without world-model rollouts, we report only its key accuracy and active accuracy; the Gap metric does not apply. For world-model methods, all three metrics are computed from the CEM-recovered action sequence. Appendix D describes the shared data alignment and evaluation protocol.

Results. ActSWM improves the action gap in all three domains, by up to $16.5\times$, and raises active accuracy by up to 0.711 while maintaining competitive overall key accuracy. G-IDM attains zero active accuracy despite moderate overall accuracy because it mainly predicts the dominant inactive class. The consistent action-gap gains demonstrate stronger action-conditioned steerability.

Across the three evaluations, metrics that explicitly compare action alternatives are more informative about planning utility than future-state similarity or overall key accuracy alone. Q1 establishes this distinction diagnostically, Q2 verifies its consequence in closed-loop control, and Q3 shows that the same action-sensitive representations support offline action inference. The shared mechanism is a latent scoring landscape that changes meaningfully with candidate actions, enabling CEM to rank alternatives rather than merely follow visual context. Thus, rollout separation serves both as a diagnostic of controllability and as a training principle for practical planning. These results establish action sensitivity as more than an auxiliary representation property: it links counterfactual rollout comparison, reliable closed-loop planning, and scalable action labeling from gameplay video.

Conclusion

We introduced ActSWM to address Context Collapse, where plausible latent rollouts lose sensitivity to action inputs. ActSWM combines multi-step prediction, alternative-action rollout separation, and a fixed action readout to preserve distinguishable, action-recoverable transitions. Across step-drift diagnostics, closed-loop planning, and offline action recovery, ActSWM improves action sensitivity and task success while retaining prediction quality. These results establish action sensitivity as a core requirement for planning-oriented latent world models beyond future-state accuracy.

References

- Alles, M.; Zhang, X.; van der Smagt, P.; and Becker-Ehmck, P. 2025. Latent Action World Models for Control with Unlabeled Trajectories. *arXiv:2512.10016*.
- Alonso, E.; Jelley, A.; Micheli, V.; Kanervisto, A.; Storkey, A.; Pearce, T.; and Fleuret, F. 2024. Diffusion for World Modeling: Visual Details Matter in Atari. In Globerson, A.; Mackey, L.; Belgrave, D.; Fan, A.; Paquet, U.; Tomczak, J.; and Zhang, C., eds., *Advances in Neural Information Processing Systems*, volume 37, 58757–58791. Curran Associates, Inc.
- Assran, M.; Duval, Q.; Misra, I.; Bojanowski, P.; Vincent, P.; Rabbat, M.; LeCun, Y.; and Ballas, N. 2023. Self-Supervised Learning from Images with a Joint-Embedding Predictive Architecture. In *2023 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 15619–15629.
- Baker, B.; Akkaya, I.; Zhokov, P.; Huizinga, J.; Tang, J.; Ecoffet, A.; Houghton, B.; Sampedro, R.; and Clune, J. 2022. Video PreTraining (VPT): Learning to Act by Watching Unlabeled Online Videos. In Koyejo, S.; Mohamed, S.; Agarwal, A.; Belgrave, D.; Cho, K.; and Oh, A., eds., *Advances in Neural Information Processing Systems*, volume 35, 24639–24654. Curran Associates, Inc.
- Bardes, A.; Garrido, Q.; Ponce, J.; Chen, X.; Rabbat, M.; LeCun, Y.; Assran, M.; and Ballas, N. 2024. Revisiting Feature Prediction for Learning Visual Representations from Video. *arXiv:2404.08471*.
- Bengio, S.; Vinyals, O.; Jaitly, N.; and Shazeer, N. 2015. Scheduled Sampling for Sequence Prediction with Recurrent Neural Networks. In Cortes, C.; Lawrence, N.; Lee, D.; Sugiyama, M.; and Garnett, R., eds., *Advances in Neural Information Processing Systems*, volume 28. Curran Associates, Inc.
- Cai, S.; Mu, Z.; He, K.; Zhang, B.; Zheng, X.; Liu, A.; and Liang, Y. 2025. MineStudio: A Streamlined Package for Minecraft AI Agent Development. *arXiv:2412.18293*.
- Chen, B.; Monso, D. M.; Du, Y.; Simchowitz, M.; Tedrake, R.; and Sitzmann, V. 2024. Diffusion Forcing: Next-token Prediction Meets Full-Sequence Diffusion. *arXiv:2407.01392*.
- de Boer, P.-T.; Kroese, D. P.; Mannor, S.; and Rubinstein, R. Y. 2007. A tutorial on the cross-entropy method. 134(1): 19–67.
- Dedieu, A.; Ortiz, J.; Lou, X.; Wendelken, C.; Guntupalli, J. S.; LeFrach, W.; Lazaro-Gredilla, M.; and Murphy, K. P. 2025. Improving Transformer World Models for Data-Efficient RL. In *Forty-second International Conference on Machine Learning*.
- Du, T.; Zhang, Q.; Wang, Y.; and Wang, Y. 2026. Beyond the Next Step: Variable-Length Latent World Models for Long-Horizon Planning. *arXiv:2606.21775*.
- Fan, L.; Wang, G.; Jiang, Y.; Mandlekar, A.; Yang, Y.; Zhu, H.; Tang, A.; Huang, D.-A.; Zhu, Y.; and Anandkumar, A. 2022. MineDojo: Building Open-Ended Embodied Agents with Internet-Scale Knowledge. *arXiv:2206.08853*.
- Hafner, D. 2021. Benchmarking the Spectrum of Agent Capabilities. In *Deep RL Workshop NeurIPS 2021*.
- Hafner, D.; Lillicrap, T.; Fischer, I.; Villegas, R.; Ha, D.; Lee, H.; and Davidson, J. 2019. Learning Latent Dynamics for Planning from Pixels. *arXiv:1811.04551*.
- Hafner, D.; Pasukonis, J.; Ba, J.; and Lillicrap, T. 2024. Mastering Diverse Domains through World Models. *arXiv:2301.04104*.
- Han, Y.; and Yilmaz, A. 2026. Enhancing Policy Learning with World-Action Model. *arXiv:2603.28955*.
- Hansen, N.; Su, H.; and Wang, X. 2024. TD-MPC2: Scalable, Robust World Models for Continuous Control. *arXiv:2310.16828*.
- Hansen, N.; Wang, X.; and Su, H. 2022. Temporal Difference Learning for Model Predictive Control. *arXiv:2203.04955*.
- Ivashkov, P.; Balestrieri, R.; and Schölkopf, B. 2026. Sensorimotor World Models: Perception for Action via Inverse Dynamics. *arXiv:2606.20104*.
- Li, J.; Wang, Q.; Wang, Y.; Jin, X.; Li, Y.; Zeng, W.; and Yang, X. 2026. Open-World Reinforcement Learning over Long Short-Term Imagination. *arXiv:2410.03618*.
- Lifshitz, S.; Paster, K.; Chan, H.; Ba, J.; and McIlraith, S. 2023. STEVE-1: A Generative Model for Text-to-Behavior in Minecraft. In Oh, A.; Naumann, T.; Globerson, A.; Saenko, K.; Hardt, M.; and Levine, S., eds., *Advances in Neural Information Processing Systems*, volume 36, 69900–69929. Curran Associates, Inc.
- Maes, L.; Lidec, Q. L.; Scieur, D.; LeCun, Y.; and Balestrieri, R. 2026. LeWorldModel: Stable End-to-End Joint-Embedding Predictive Architecture from Pixels. *arXiv:2603.19312*.
- Peng, W.; and Molin, H. 2026. Temporal Discriminative World Models. Project page; paper in progress.
- Tan, W.; Li, X.; Fang, Y.; Yao, H.; Yan, S.; Luo, H.; Ao, T.; Li, H.; Ren, H.; Yi, B.; Qin, Y.; An, B.; Liu, L.; and Shi, G. 2025. Lumine: An Open Recipe for Building Generalist Agents in 3D Open Worlds. *arXiv:2511.08892*.
- Vuong, A. D.; Vo, T. V.; Sohail, A.; Ding, H.; Ma, L.; Liang, X.; Duan, A.; Laptev, I.; and Reid, I. 2026. World2Act: Latent Action Post-Training from World Model Dynamics. *arXiv:2603.10422*.
- Wang, G.; Xie, Y.; Jiang, Y.; Mandlekar, A.; Xiao, C.; Zhu, Y.; Fan, L.; and Anandkumar, A. 2023. Voyager: An Open-Ended Embodied Agent with Large Language Models. In *Intrinsically-Motivated and Open-Ended Learning Workshop @NeurIPS2023*.
- Wang, Y.; Zhang, F.; Zhan, D.-C.; Zhao, L.; Wang, K.; and Bian, J. 2026. Co-Evolving Latent Action World Models. *arXiv:2510.26433*.
- Wang, Z.; Cai, S.; Liu, A.; Jin, Y.; Hou, J.; Zhang, B.; Lin, H.; He, Z.; Zheng, Z.; Yang, Y.; Ma, X.; and Liang, Y. 2025a. JARVIS-1: Open-World Multi-Task Agents With Memory-Augmented Multimodal Language Models. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 47(3): 1894–1907.
- Wang, Z.; Cai, S.; Mu, Z.; Lin, H.; Zhang, C.; Liu, X.; Li, Q.; Liu, A.; Ma, X.; and Liang, Y. 2024. OmniJARVIS: Unified Vision-Language-Action Tokenization Enables Open-World

Instruction Following Agents. In *Multi-modal Foundation Model meets Embodied AI Workshop @ ICML2024*.

Wang, Z.; Li, X.; Ye, Y.; Fang, J.; Wang, H.; Liu, L.; Liang, S.; Lu, J.; Wu, Z.; Feng, J.; Zhong, W.; Li, Z.; Wang, Y.; Miao, Y.; Zhou, B.; Li, Y.; Wang, H.; Zhao, Z.; Wu, F.; Jiang, Z.; Tan, W.; Yao, H.; Yan, S.; Li, X.; Liang, Y.; Qin, Y.; and Shi, G. 2025b. Game-TARS: Pretrained Foundation Models for Scalable Generalist Multimodal Game Agents. arXiv:2510.23691.

Yan, P.; Abdulkadir, A.; Schatte, G. A.; Aguzzi, G.; Gha, J.; Pascher, N.; Rosenthal, M.; Gao, Y.; Grewe, B. F.; and Stadelmann, T. 2025. Learning Actionable World Models for Industrial Process Control. arXiv:2503.01411.

Yuan, H.; Zhang, C.; Wang, H.; Xie, F.; Cai, P.; Dong, H.; and Lu, Z. 2023. Skill Reinforcement Learning and Planning for Open-World Long-Horizon Tasks. In *NeurIPS 2023 Foundation Models for Decision Making Workshop*.

Zhang, W.; Terver, B.; Zholus, A.; Chitnis, S.; Sutaria, H.; Assran, M.; Balestriero, R.; Bar, A.; Bardes, A.; LeCun, Y.; and Ballas, N. 2026a. Hierarchical Planning with Latent World Models. arXiv:2604.03208.

Zhang, Y.-G.; Du, T.; Zhang, Q.; and Wang, Y. 2026b. DWM: Separating World Effects from Actions in Latent World Models. arXiv:2607.18715.

Zhang, Z.; Wang, Y.; Guan, Z.; Yang, Y.; Shi, B.; Zong, T.; Yi, H.; Chao, G.; Chen, X.; Yang, T.; Bao, C.; Yu, T.; Zhou, J.; and Xu, J. 2026c. Delta-JEPA: Learning Action-Sensitive World Models via Latent Difference Decoding. arXiv:2606.31232.

Zhou, G.; Pan, H.; LeCun, Y.; and Pinto, L. 2025. DINO-WM: World Models on Pre-trained Visual Features enable Zero-shot Planning. arXiv:2411.04983.

A Model Architecture and Training Configuration

This section lists the architecture and optimization settings used for the experiments in the main text. Actions are aggregated with `frameskip=4`, so one world-model step corresponds to four original environment steps. Table 2 summarizes the shared model and training configuration.

Table 3 lists the hyperparameters corresponding to the objective defined in the method section.

The implementation retains the configuration key `idm.stop_grad=true`. Despite its name, this option only excludes the action readout parameters ϕ_0 from the optimizer; it does not detach the latent inputs. The readout loss therefore backpropagates through encoded and rollout-predicted transitions to update the encoder and predictor. In the jointly trained readout ablation, ϕ is added to the optimizer while all other settings remain unchanged.

B Step-Drift Evaluation Protocol

Table 4 provides the implementation settings for the step-drift evaluation. Both rollouts use the same context and world model and differ only in their future action inputs.

Table 2: Model and training configuration.

Category	Item	Value
Data	frameskip	4
Data	segment length $H + K$	44
Data	split ratio	90/5/5%
Image	input resolution	128 px
Image	patch size	16
Encoder	backbone	ViT-tiny
Latent	embed dim	192
Predictor	depth / heads	6 / 16
Predictor	MLP hidden dim	2048
Predictor	action conditioning	AdaLN
Training	batch size	32
Training	optimizer	AdamW
Training	learning rate	1×10^{-4}
Training	weight decay	1×10^{-3}
Training	max steps	100k
Training	precision	mixed 16-bit
Training	gradient clipping	1.0
Training	random seed	3072

Table 3: Loss and objective hyperparameters.

Loss Term	Parameter	Value
Rollout loss	context length H	32
Rollout loss	rollout horizon K	12
SigReg	weight λ_{sig}	0.09
SigReg	knots / num proj	17 / 1024
Hinge loss	weight λ_{hinge}	0.5
Hinge loss	margin m	0.3
Action readout	weight λ_{readout}	1.0
Action readout	predicted-transition weight	1.0
	α_{pred}	
Action readout	hidden dim	512
Action readout	trainable	no

For each valid window, we encode the first H observations once and autoregressively generate two K_{eval} -step rollouts from that shared context. The recorded-action rollout uses the aligned dataset controls, while the counterfactual rollout replaces every future control with the all-zero vector. At each step, we compute cosine similarity to the same encoded future target and average s_k^{gt} , s_k^0 , and $\Delta_k = s_k^{\text{gt}} - s_k^0$ over all valid windows. Windows crossing an episode boundary or invalid-frame marker are excluded. All model variants use this identical sampling and aggregation procedure.

Table 5 reports the step-31 prediction quality and action-gap values.

The short-context and high-resolution rows isolate context length and input resolution, while the +Rollout rows use multi-step training. The jointly trained readout ablation optimizes the readout together with the world model; ActSWM instead uses the frozen readout described in Appendix A. Thus, the reported differences arise from the specified model or objective change rather than from the evaluation protocol.

Table 4: Step-drift evaluation configuration.

Item	Value
context length H	32
evaluation horizon K_{eval}	32
frameskip	4
batch size	64
max sampled windows	5000
effective count per step	1280
random seed	1234

Table 5: Step-31 prediction quality and action gap. GT denotes the rollout conditioned on dataset actions, and Zero denotes the rollout conditioned on all-zero actions.

Method	GT	Zero	Gap
LeWM, $H = 3$, 128px	0.239	0.244	-0.005
LeWM, $H = 32$, 128px	0.816	0.815	0.001
LeWM, $H = 3$, 256px, +Rollout	0.810	0.810	0.000
LeWM, $H = 32$, 128px, +Rollout	0.972	0.970	0.002
LeWM, +Rollout, +Joint Readout	0.698	0.106	0.592
ActSWM, ours	0.923	0.163	0.760

C CEM Planning and Task Setup

This section provides the implementation details for the closed-loop planning experiment in Q2. LeWM and ActSWM use the same planner, action-block library, sampling distributions, target latent sequences, and execution rule; only the world model used for rollout scoring differs. Table 6 lists the shared planning configuration.

Table 6: Global CEM/MPC planning configuration.

Item	Value
CEM candidates	512
CEM iterations	6
elite fraction	0.125
rollout batch size	512
frameskip	4
action-block library	Table 7
planning horizon K_{plan}	12
execution	MPC, execute 1 chunk
camera dims	Gaussian sampling
button dims	Bernoulli sampling

Candidate action library. In the Minecraft planning experiments, CEM searches over task-specific composite action chunks rather than unconstrained raw actions. Each action chunk contains 4 raw environment steps. Unless otherwise specified, the same atomic action is repeated for all 4 steps. Pitch and yaw denote camera movements, while attack, use, jump, forward, and back denote pressed buttons. The prior weights are used to sample candidate chunks before CEM updates the search distribution using elite samples.

Table 8 reports the task-specific success condition, observation mask, planning budget, environment-step budget, and

Table 7: Task-specific candidate action chunks used by the CEM planner in Minecraft planning experiments.

Task	Chunk	Action composition	Weight
Mine Stone	look down	pitch +8°	0.15
	mine down	attack + pitch +82°	0.55
	forward	forward	0.20
	back	back	0.10
Place Torch	turn right	yaw +3°	0.20
	forward	forward	0.20
	look down	pitch +6°	0.20
	wait	no-op	0.15
	place	use + pitch +28°	0.25
Build Pillar	stack action	look down, look down, jump, use + pitch +45°	0.50
	forward	forward	0.15
	back	back	0.15
	turn left	yaw −5°	0.10
	turn right	yaw +5°	0.10

success count. All tasks use $K_{\text{plan}} = 12$.

The *Trigger* column specifies the success signal: torch placement and stone mining use environment events, whereas pillar building is judged from the completed structure. The two values in *Plan-env* are the planning and environment-step budgets, respectively. Each method is evaluated on the same 20 initial conditions and reference trajectories.

Table 8: Task-level settings and success counts.

Task	Trigger	Mask	Plan-env	LeWM	ActSWM
Place Torch	use_item:torch	mining	50–100	19/20	20/20
Mine Stone	mine_block	mining	150–300	10/20	19/20
Build Pillar	human_label	none	50–40	11/20	17/20

D Cross-Game CEM Evaluation Protocol

This section details the shared evaluation protocol for the three-game action-recovery experiment (Q3). We sample 15 episode-aware windows per game, each spanning $K_{\text{rec}} = 32$ model steps with frameskip 4 and a target offset of $\delta = 128$ raw frames, equal to 32 model steps. Inputs are resized to 128×128 , and actions are represented by 14 binary controls using the 56-dimensional four-step sliding-window encoding. The controls comprise 10 buttons and four discretized camera directions. ActSWM uses $H = 32$, whereas LeWM uses its native context length $H = 3$.

For each recovered step, CEM uses 1024 candidates, 128 elites, and 8 iterations. The first step is initialized from $\mathcal{N}(\mu_{\text{gt}}, \sigma_{\text{gt}})$; subsequent steps are centered at the previous recovered action with standard deviation $0.5\sigma_{\text{gt}}$. The first CEM iteration mixes 25% empirical action-pool samples with 75% Gaussian samples to anchor the search in valid key combinations; subsequent iterations sample purely from the Gaussian to refine around the elite mean. The random baseline draws an i.i.d. sequence from the empirical action pool. All experiments are run with `-human-guide` (other initialization flags such as `-gt-dist` and `-zero-init` are

Reference Trajectories for MineStudio Planning Tasks

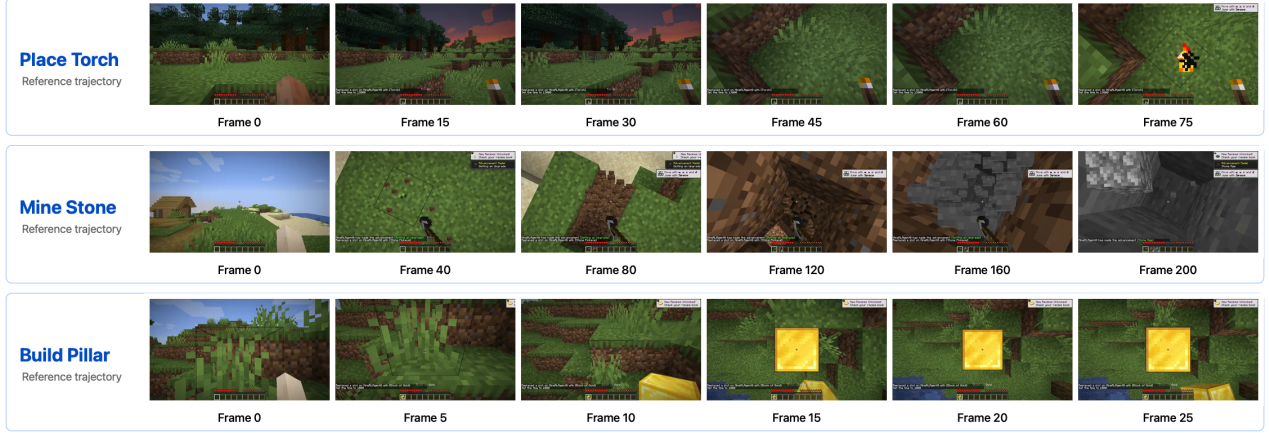


Figure 5: Reference trajectories for the three Minecraft planning tasks. Each row shows the target video segment for one task and marks the selected reference frames; these clips are encoded into target latent-embedding sequences for planning.

not used), together with `-img-size 128` to ensure uniform input resolution. We report action gap, key accuracy, and active accuracy; D2E G-IDM is evaluated only on the two accuracy metrics because it does not use world-model search.

Evaluation windows are sampled with episode-boundary awareness: a candidate start index is valid only if neither the start frame nor the target frame crosses an episode boundary, both frames belong to the same episode, and both are marked valid. From the valid starts, 15 windows are randomly sampled with seed 42.

All methods are evaluated on the same cleaned windows and binary key dimensions. For world-model methods, key accuracy is computed from the CEM-recovered action sequence. For D2E G-IDM, it is computed directly from the predicted action labels.

E Fixed Action-Readout Separation Argument

This section expands the separation argument used in the method section. Let u_i and u_j be two latent transitions with corresponding actions a_i and a_j . Assume the fixed action readout q_{ϕ_0} is locally L -Lipschitz on the region containing these transitions:

$$\|q_{\phi_0}(u_i) - q_{\phi_0}(u_j)\| \leq L\|u_i - u_j\|. \quad (17)$$

If the readout recovers the two actions with errors bounded by ϵ_i and ϵ_j ,

$$\|q_{\phi_0}(u_i) - a_i\| \leq \epsilon_i, \quad \|q_{\phi_0}(u_j) - a_j\| \leq \epsilon_j, \quad (18)$$

then the triangle inequality gives

$$\|a_i - a_j\| \leq \epsilon_i + \|q_{\phi_0}(u_i) - q_{\phi_0}(u_j)\| + \epsilon_j. \quad (19)$$

Combining this inequality with Lipschitz continuity yields

$$\|u_i - u_j\| \geq \frac{\|a_i - a_j\| - \epsilon_i - \epsilon_j}{L}. \quad (20)$$

Therefore, if different actions are decoded accurately by a fixed readout, their latent transitions must remain separated by a positive margin whenever $\|a_i - a_j\| > \epsilon_i + \epsilon_j$. This argument does not claim that random initialization is optimal; rather, it explains why freezing the readout provides a stable constraint on latent transition representations. A jointly trained readout does not provide the same constraint because q_{ϕ} can move its decision boundary as the representation changes, reducing action-prediction loss without necessarily increasing separation between transitions associated with different actions.

F Gameplay Training Data Cleaning Pipeline

This section describes the alignment and cleaning pipeline used for paired gameplay recordings in the cross-game evaluation. It ingests MCAP action streams and corresponding MKV videos and produces HDF5 clips with ground-truth 14-dimensional binary control labels. Public videos without native action logs follow the separate construction pipeline in Appendix G before model-based action annotation.

Stage 1: Rule-based prefiltering. Raw action streams from the input MCAP are processed frame-by-frame. Frames where keyboard or mouse signals fail to decode, contain no input events, or exhibit timestamp anomalies are discarded. Mouse movements are converted to binary turn/look controls via a threshold of 3 pixels per frame. For sources with explicit death records (e.g., CS deathmatch), frames within a fixed time window after each death event are also removed at this stage. The prefiltered frames are segmented into sub-episodes by splitting at PTS gaps exceeding 150 ms, discarding segments shorter than 200 ms. Valid frames are then mapped via the action projection module to a 14-dimensional binary control vector (buttons: W, A, S, D, Shift, Ctrl, Fire, Aim, Reload, Jump; camera: Turn_L, Turn_R, Look_U, Look_D). Frames within 3 frames of a segment boundary are marked as `frame_valid=0` to prevent evaluation windows from

Data Cleaning Pipeline

Dirty Raw Game Recording
TO Training-Ready Data

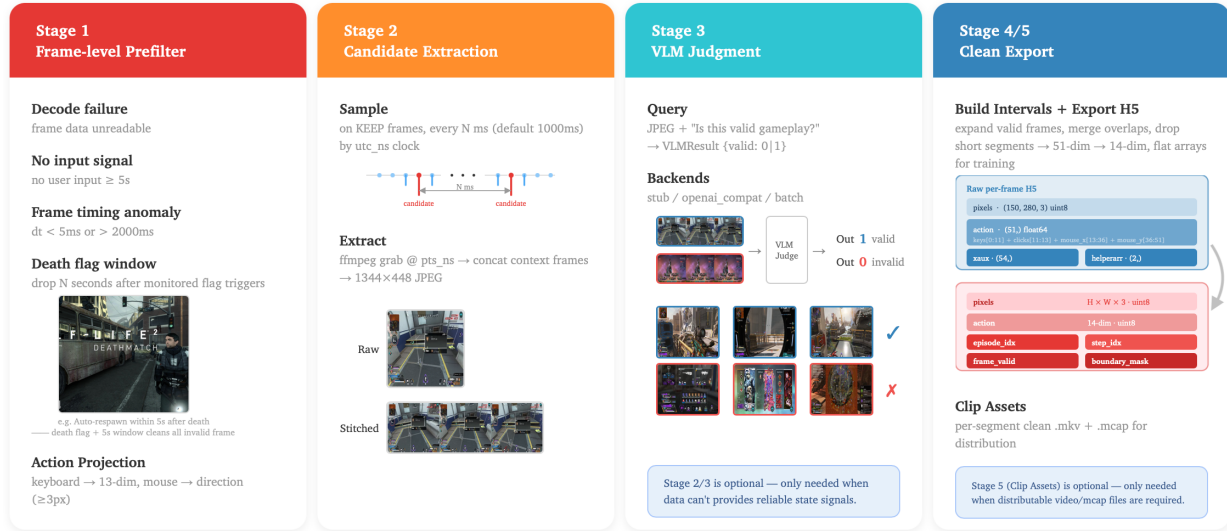


Figure 6: Overview of the gameplay data cleaning pipeline.

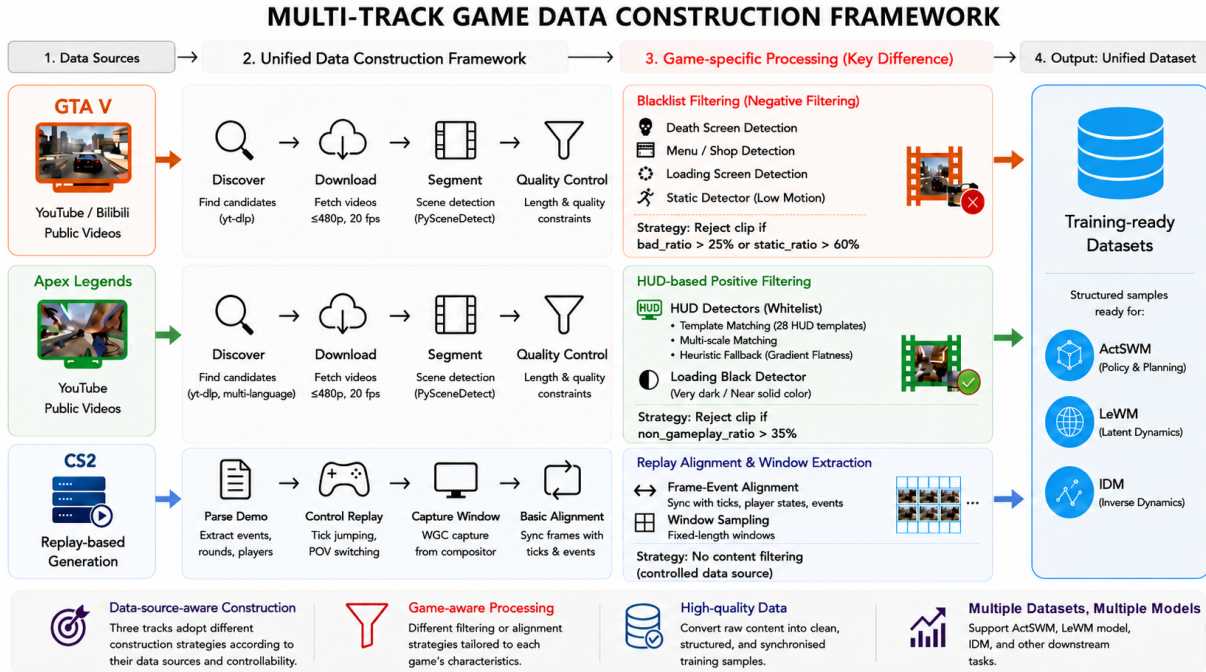


Figure 7: Multi-track game data construction framework. GTA V and Apex share web-video discovery, segmentation, and quality-control stages but use blacklist and HUD-whitelist filtering, respectively. CS2 instead uses controlled demo replay, WGC capture, and frame–event alignment. All pipelines produce a unified training-ready data format.

crossing PTS-discontinuity boundaries.

Stage 2–3: Candidate extraction and VLM judging.
For domains where non-gameplay states (menus, loading

screens, spectator views) are common, candidate frames are extracted from the prefiltered stream at a sampling interval of 1s and submitted to a VLM judge. The VLM classifies each frame as valid gameplay or non-gameplay using a domain-specific prompt, operating in either online mode (per-frame API calls with thread-pool concurrency) or batch mode (batch file submission for reduced cost). Consecutive valid frames separated by gaps under 150 ms are merged into contiguous intervals; intervals shorter than 200 ms are discarded. The resulting valid intervals define the final export boundaries.

Stage 4–5: HDF5 export and post-processing. Frames within the validated intervals are read from the source video via ffmpeg (decoded at the source frame rate with pixel format conversion) and written into an HDF5 file containing six datasets: `pixels` (video frames), `action` (14-dim binary key labels), `episode_idx`, `step_idx`, `frame_valid`, and `boundary_mask`. Episode boundaries are marked to prevent cross-episode window sampling during evaluation. An optional Stage 5 crops and exports cleaned video and MCAP segments for downstream inspection; for H5-mode pipelines, Stage 5 outputs per-stage summary statistics instead.

G Model-annotated Dataset Construction Pipeline

Figure 7 presents three game-specific data pipelines: GTA V, Apex Legends, and Counter-Strike 2 (CS2). Although their sources differ, all three pipelines convert raw content into clean, structured first-person gameplay clips suitable for subsequent model annotation. We factor the work into two algorithms. Algorithm 2 covers the shared public-video construction skeleton of GTA and Apex; the *key difference* is the domain filter F_d —negative blacklist filtering for GTA vs. positive HUD filtering for Apex. Algorithm 3 covers CS2, which generates video from structured demos via tick-aligned engine control, GPU-compositor capture, and frame–event alignment (no content filtering; the source is already controlled). Paired recordings used for quantitative evaluation retain native controls through Appendix F; clips without action logs can instead be annotated with the recovery procedure in Appendix D. In the main text and result tables, CS2 is abbreviated as CS.

Algorithm 2 formalizes the shared stages of video discovery, download, segmentation, domain-specific filtering, and export. The filtering strategy is the key difference: GTA uses *negative* blacklist filtering over death, menu, loading, and static states, rejecting clips with a high unwanted-state ratio. Apex instead uses *positive* multi-scale HUD whitelist matching because motion alone is unreliable in combat. Algorithm 3 generates tick-aligned synchronized training samples from structured demos via engine control, WGC GPU-compositor capture rather than GDI framebuffer capture, and `FrameEventAlign` (implementation details in the supplementary software).

The filtering rules are high-throughput construction heuristics rather than standalone detector benchmarks. For GTA V, clips are rejected when the bad-state ratio exceeds

Algorithm 2: Public Video Gameplay Model-annotated dataset Construction Pipeline (GTA V / Apex Legends)

Require: Domain $d \in \{\text{GTA}, \text{Apex}\}$; keywords kw ; optional seed URLs; thresholds τ_d
Ensure: Clean gameplay clips V_d for model annotation (mute, fixed fps/res.)

```

1:  $S_d \leftarrow \text{Crawl}(\text{kw})$  {discover source videos}
2:  $S_d \leftarrow \text{Download}(S_d)$  {optional seeds mainly for GTA}
3:  $S_d \leftarrow \text{Norm}(S_d)$  {normalize source videos}
4:  $C_d \leftarrow \text{SceneCut}(S_d)$  {Segment: short clip candidates}
5:  $\mathcal{B} \leftarrow \{\text{death, menu, load, static}\}$ 
6: for  $c \in C_d$  do
7:   if  $d=\text{GTA}$  then
8:      $\rho \leftarrow \text{Blacklist}(c; \mathcal{B})$  {negative filtering}
9:      $\mathbf{1}_{\text{keep}} \leftarrow [\rho \leq \tau_{\text{GTA}}]$  {reject if bad-state ratio high}
10:  end if
11:  if  $d=\text{Apex}$  then
12:     $e \leftarrow \text{HUDMatch}_{\text{ms}}(c)$  {positive filtering; multi-scale/layout}
13:     $\ell \leftarrow \text{LoadingDetect}(c)$  {reject dark/near-uniform transitions}
14:     $\mathbf{1}_{\text{keep}} \leftarrow [e \geq \tau_{\text{Apex}} \wedge \ell = \text{false}]$  {require play HUD; drop loads}
15:  end if
16:  if  $\mathbf{1}_{\text{keep}}$  then
17:     $V_d \leftarrow V_d \cup \{\text{MuteExport}(c)\}$ 
18:  end if
19: end for
20: return  $V_d$ 
```

Algorithm 3: Replay-based Gameplay Data Generation Pipeline (CS2)

Require: Legal demos $\mathcal{D}$ (`.dem/.dem.zst`); HLAE/CFG stack; WGC; FFmpeg
Ensure: Tick-aligned gameplay clips V_{CS2} for model annotation (mute, fixed fps/res.)

```

1:  $M \leftarrow \text{Parse}(\mathcal{D})$  {players, rounds, ticks, kills, account IDs}
2:  $C \leftarrow \{(p, r, [t_0, t_1])\}$  {player×round alive-only windows}
3: for  $(p, r, [t_0, t_1]) \in C$  do
4:    $\text{cfg} \leftarrow \text{Jump}(t_0) \circ \text{Spec}_1(p)$  {seek and switch POV}
5:    $\text{cfg} \leftarrow \text{cfg} \circ \text{StripHUD} \circ \text{Quit}(t_1)$ 
6:    $\text{Launch}(\text{HLAE}, \text{Play}(M, \text{cfg}))$  {controlled replay}
7:    $f \leftarrow \text{WGC}(\cdot; [t_0, t_1])$  {GPU compositor capture}
8:    $c \leftarrow \text{FrameEventAlign}(f, M)$  {align ticks, states, and events}
9:    $v \leftarrow \text{MuteExport}(c)$  {full_round.mp4}
10:   $V_{\text{CS2}} \leftarrow V_{\text{CS2}} \cup \{v\}$ 
11: end for
12: return  $V_{\text{CS2}}$ 
```

25% or the static ratio exceeds 60%; approximately 40% of segmented footage is retained, with a target of roughly 250 hours. Apex uses 28 multi-scale HUD templates, rejects clips whose non-gameplay ratio exceeds 35%, and disables static-

motion rejection. CS2 uses controlled replay without content filtering and targets roughly 100 hours of synchronized first-person video. We manually inspected sampled retained and rejected clips to verify the resulting data quality.